

Task 1.1

I learned how to store a string in a variable and display it with `print()`. This is the foundation for everything else: without storing the text in a variable, I wouldn't be able to manipulate it in the following tasks.

```
sentence = "EPITECH is a great school!"  
print(sentence)
```

Task 1.2 and 1.3

I learned that in Python every character in a string has an index, starting at 0. I used `string[0]` to get the first character and `string[-1]` to get the last one — negative indexing counts from the end, so I don't need to know the string's length to grab the last character.

```
sentence = "EPITECH is a great school!"  
print(sentence[0])
```

```
sentence = "EPITECH is a great school!"  
print(sentence[-1])
```

Task 1.4

I learned to use slicing (`string[5:10]`) to grab a chunk of the string at once. The first number is where it starts (included) and the second is where it stops (not included), so to get the 5th through the 10th character I used indexes 5 and 10.

```
sentence = "EPITECH is a great school!"  
print(sentence[5:10])
```

Task 1.5

I used the `.lower()` method, which already converts any string to lowercase without needing loops.

```
sentence = "EPITECH is a great school!"  
print(sentence.lower())
```

Task 1.6

I learned how to replace a substring inside another string using a manual `for` loop, I went through the text character by character, always comparing the next two characters with "tu"; when I found a match, I added "ta" to the result and skipped the next position so I wouldn't reprocess part of the "tu" I had just replaced.

```
text = "tutu on the tuki-kata"
result = ""
pular = 0

for i in range(len(text)):
    if pular > 0:
        pular -= 1
        continue
    if text[i:i+2] == "tu":
        result += "ta"
        pular = 1
    else:
        result += text[i]

print(result)
```

Task 1.7

I learned that the `.find()` method searches for a substring and returns the index where it starts, or `-1` if nothing is found unlike other languages, it doesn't raise an error when the search fails.

```
string = "Hello world!"
position = string.find("a")
print(position)
#this code just says to you the position of an expecific char, and if this char is not on the frase
#you will see a -1 position
```

Task 1.8 and 1.9

This was the most complex one: I learned how the `[start:stop:step]` slicing syntax works with all its parameters, including a negative step (which reverses the reading direction). I broke the complex code into separate steps to visualize what each slice was doing to the result of the previous one, which helped me understand that you can chain several slices in a row.

```
p = "abcdefghij"
print(p[::-2][:5][::-1][3:])
```

```
# [start:stop:step]
# Sentence = abcdefghj

# p[::-2] → scrolls backwards jumping from 2 to 2: "jhfdb" (will take de char 9,7,5,3,1)
# [:5] → get the first 5 (there are already only 5): "jhfdb"
# [::-1] → invert: "bdfhj"
# [3:] → from index 3 to the end: "hj"
```

Task 1.10

I learned that I could achieve the result with a `for` loop and `range(10)`, which simply runs the `print(text)` line ten times.

```
text = "Hello World!"
for i in range(10):
    print(text)
```

Task 1.11

I learned that Python is a strongly typed language: you can't add a `str` and an `int` directly with `+`, you have to convert the number to a string first with `str()`, or use f-strings, which handle that conversion automatically.

```
print("hello" + 42)
#TypeError, because you are trying to concatenate a string with an integer, and you can't
#do this in pyhton
```

Challenge

I combined several concepts: I used `.lower()` to ignore uppercase/lowercase, `.count()` to count occurrences without worrying about overlaps, and `[::-1]` to also count the reversed word. This showed me how small language tools can be combined to solve a more complex problem.

```
def contar_ocorrencias(text):  
    text = text.lower()  
    words = ["cat", "garden", "mice"]  
    total = 0  
    for Word in words:  
        total += text.count(Word)  
        total += text.count(Word[::-1])  
    return total  
  
text = input("Type a text: ")  
print(contar_ocorrencias(text))
```

Task 3.1

I learned to use the `.capitalize()` method, which automatically capitalizes only the first letter.

```
name = input("What's your name? ")  
print(f"Hello {name.capitalize()}")
```

Task 3.2

I understand that `input()` always returns a string, even if the person types a number — so it's always necessary to manually convert it with `int()` or `float()` before doing math. I also used `try/except` to automatically try to detect whether the typed value was an `int`, a `float`, or just plain text, catching the `ValueError` when the conversion failed.

```
number = input("Type a number: ")
try:
    value = int(number)
except ValueError:
    try:
        value = float(number)
    except ValueError:
        value = number

print(type(value))
```

Task 3.3

I applied what I learned in the previous task, converting the two typed numbers with `int()` before adding them, and used an f-string to build the formatted output sentence.

```
number1 = int(input("Type the first number: "))
number2 = int(input("Type the second number: "))

print(f"The sum of the numbers is: {number1 + number2}")
```

Task 3.4

I learned to use `.split()` to break a sentence into a list of words (splitting on spaces), and then used a generator expression with `.join()` to grab the first letter of each word and join them into a single word, without needing explicit loops.

```
frase = input("Type a frase: ")
acronym = "".join(word[0] for word in frase.split())
print(acronym)
```

Task 3.5

I learned to use dictionaries to count the frequency of elements (in this case, letters), using the `.get(key, default)` method to avoid errors when the key doesn't exist yet. I also learned to build a simple comparison between the text's letter distribution and reference frequency tables per language, computing the sum of squared differences to find the closest matching language this was my first time implementing a sort of simple "classifier."

```
1  text = input("Enter a text: ")
2
3  def letter_frequency(text):
4      text = text.lower()
5      frequency = {}
6      for letter in text:
7          if letter.isalpha():
8              frequency[letter] = frequency.get(letter, 0) + 1
9
10     total = sum(frequency.values())
11     return {letter: count / total * 100 for letter, count in frequency.items()}
12
13  LANGUAGES = {
14      "portuguese": {"e": 12, "a": 13.5, "i": 6.2, "n": 5.0, "o": 10.7, "r": 6.5, "s": 7.8, "t": 4.3,
15                    "l": 5, "d": 5.0, "u": 5, "m": 4.7, "c": 3.9, "p": 2.5},
16
17      "dutch": {"e": 15, "a": 9.5, "i": 6.5, "n": 8, "o": 6.1, "r": 6.4, "s": 3.7, "t": 6.8,
18              "l": 3.6, "d": 5.9, "u": 2.0, "m": 2.2, "c": 1.2, "p": 1.6},
19
20      "german": {"e": 10, "a": 8.9, "i": 8.2, "n": 10, "o": 7.6, "r": 4.7, "s": 4.3, "t": 3.9,
21               "l": 4, "d": 3.3, "u": 2.5, "m": 2.8, "c": 3.9, "p": 3.1},
22  }
23
24  def guess_language(text_frequency):
25      best_language = None
26      smallest_difference = None
27
28      for language, reference_frequency in LANGUAGES.items():
29          difference = sum(
30              (text_frequency.get(letter, 0) - percentage) ** 2
31              for letter, percentage in reference_frequency.items()
32          )
33          if smallest_difference is None or difference < smallest_difference:
34              smallest_difference = difference
35              best_language = language
36      return best_language
37
38
39  frequency = letter_frequency(text)
40  for letter, percentage in sorted(frequency.items(), key=lambda x: -x[1]):
41      print(f"{letter} = {percentage:.1f}%")
42  print(f"Probable language: {guess_language(frequency)}")
```

Task 3.6

I learned the difference between reading text directly from `input()` (which is already UTF-8 by default in Python 3) and reading from a file, where you need to specify `encoding="utf-8"` in `open()` to make sure accents and special characters are read correctly, plus using `with open(...) as file:` to make sure the file closes automatically

```
1  def letter_frequency(text):
2      """
3      freq = {}
4      for letter in text:
5          if letter.isalpha():
6              freq[letter] = freq.get(letter, 0) + 1
7      total = sum(freq.values())
8      return {letter: count / total * 100 for letter, count in freq.items()}
9
10  LANGUAGES = {
11      "portuguese": {"e": 12, "a": 13.5, "i": 6.2, "n": 5.0, "o": 10.7, "r": 6.5, "s": 7.8, "t": 4.3,
12                    "l": 5, "d": 5.0, "u": 5, "m": 4.7, "c": 3.9, "p": 2.5},
13      "dutch": {"e": 15, "a": 9.5, "i": 6.5, "n": 10.0, "o": 6.1, "r": 6.4, "s": 3.7, "t": 6.8,
14               "l": 3.6, "d": 5.9, "u": 2.0, "m": 2.2, "c": 1.2, "p": 1.6},
15      "german": {"e": 7.6, "a": 8.9, "i": 8.2, "n": 5.5, "o": 7.6, "r": 4.7, "s": 4.3, "t": 3.9,
16                "l": 2.1, "d": 3.3, "u": 2.5, "m": 2.8, "c": 3.9, "p": 3.1},
17  }
18
19  def guess_language(text_freq):
20      best_language = None
21      smallest_difference = None
22      for language, reference_freq in LANGUAGES.items():
23          difference = sum((text_freq.get(letter, 0) - pct) ** 2 for letter, pct in reference_freq.items())
24          if smallest_difference is None or difference < smallest_difference:
25              smallest_difference = difference
26              best_language = language
27      return best_language
28
29  path = input("Enter the file path: ")
30  with open(path, "r", encoding="utf-8") as file:
31      text = file.read()
32
33  freq = letter_frequency(text)
34  for letter, pct in sorted(freq.items(), key=lambda x: -x[1]):
35      print(f"{letter} = {pct:.1f}%")
36
37  print(f"Probable language: {guess_language(freq)}")
```